



Lab of Algorithm Design

Assignment 2

Black-Box Sorting Algorithm Performance Analysis

Wilkend AUGUSTE

Student ID: 411321344

March 10, 2026

1. Introduction

Purpose: analyze black-box sorting algorithms

This experiment measures the execution time of four black-box sorting algorithms AlgoA, AlgoB, AlgoC, AlgoD provided in blackbox.dll. Since the internal code is hidden, we infer each algorithm's time complexity from how its runtime grows as input size increases.

2. Experimental Setup

- Windows 11 Home
- Intel (R) N11
- 8 GB RAM
- C++
- Compiler: MSYS2 UCRT64, MinGW-w64 g++ 64-bit, flag -O2
- Timing: `std::chrono::high_resolution_clock`
- Matlab and Python

3. Methodology

Test Data Generation

To test the algorithms, I generated arrays in five different patterns: **random**, **sorted**, **reverse**, **nearly sorted**, and **many duplicates**. The goal was to see how each algorithm behaves depending on what kind of data it receives.

Input Sizes

I didn't use the same input sizes for all four algorithms because AlgoB is extremely slow. At $n = 80$ it was already taking hundreds of milliseconds, and anything beyond that was just not realistic to test. On top of that, AlgoB only managed to complete successfully on the **sorted** pattern, every other pattern either failed or took way too long for $n \geq 15$. So I limited AlgoB to $n = 4, 15, 20, 50$, and 80 .

For AlgoA, AlgoC and AlgoD, I used much larger sizes: $n = 100, 300, 600, 1,500, 3,000, 5,000, 15,000$, and $25,000$, across all five patterns.

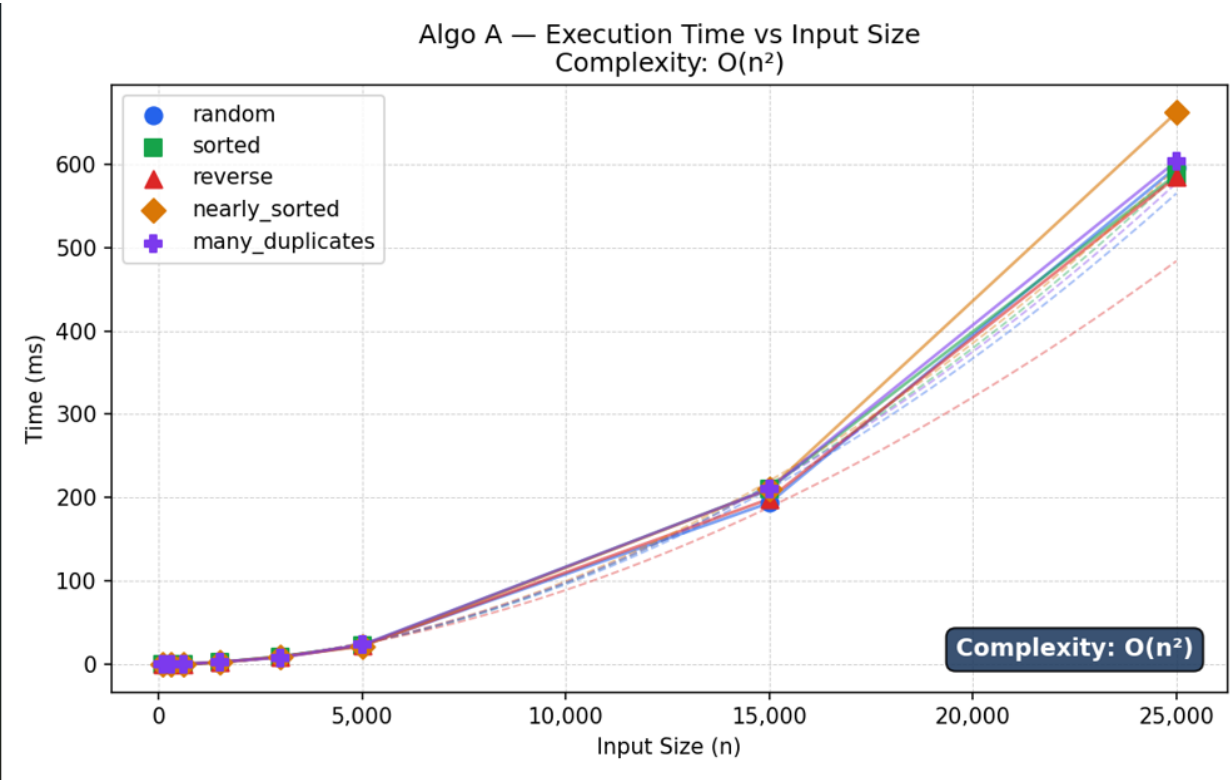
Timing

Each combination of algorithm, input size, and data pattern was run **5 times**, and I took the average of those runs as the final time in milliseconds. I also made sure to generate a fresh array before each run so that no algorithm was accidentally running on an already-sorted array from a previous test.

4. Results

algo	A
------	---

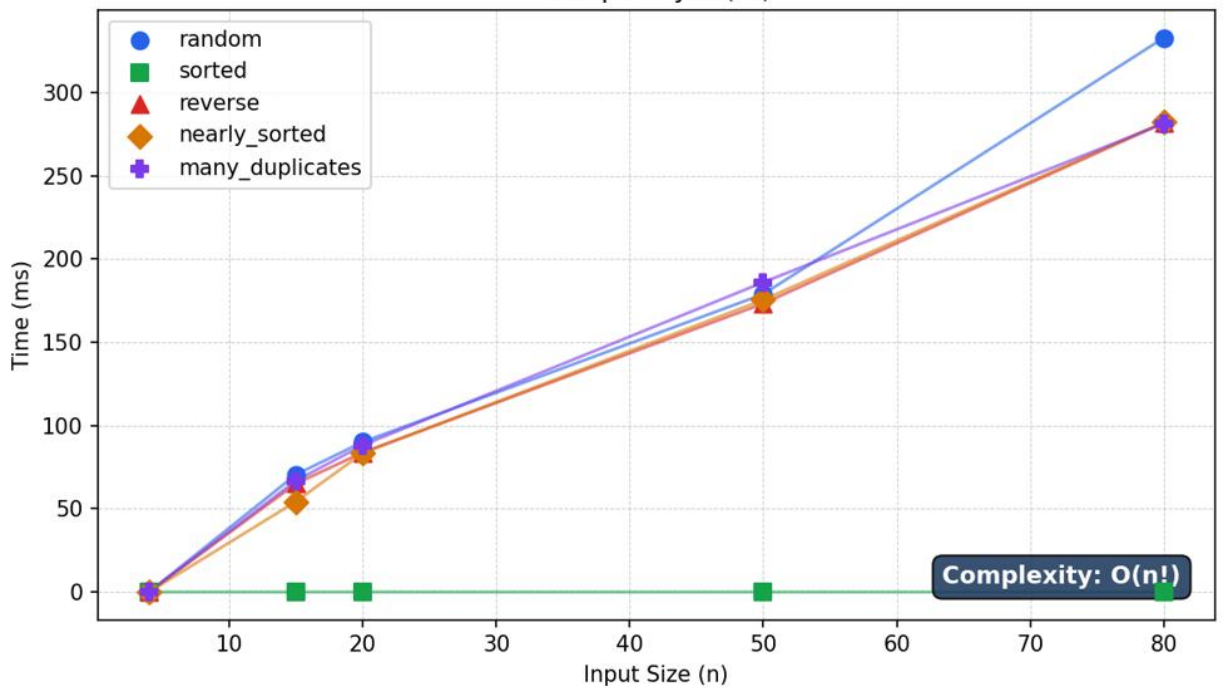
Sum of time_ms	Column Labels					
Row Labels	many_duplicates	nearly_sorted	random	reverse	sorted	Grand Total
100	0.0115	0.01056	0.01136	0.024	0.00976	0.06718
300	0.13586	0.1405	0.11614	0.11874	0.13236	0.6436
600	0.40604	0.42976	0.48064	0.48062	0.3849	2.18196
1500	3.12352	2.50306	2.55352	2.30998	2.57922	13.0693
3000	8.30168	9.31086	9.64194	9.04886	9.43076	45.7341
5000	23.4085	21.8909	22.6199	22.227	22.9329	113.0792
15000	210.477	210.161	193.694	198.462	210.543	1023.337
25000	603.561	661.926	596.361	584.759	587.596	3034.203
Grand Total	849.4251	906.37264	825.4785	817.4302	833.6089	4232.31534



algo	B
------	---

Sum of time_ms	Column Labels					Grand
Row Labels	many_duplicates	nearly_sorted	random	reverse	sorted	Total
4	0.0009	0.0011	0.0019	0.0011	0.0006	0.0056
15	66.8515	54.2957	70.5121	65.072	0.0003	256.7316
20	88.1509	83.3855	90.1121	83.8024	0.0004	345.4513
50	186.046	175.491	178.901	173.279	0.0004	713.7174
80	281.4	282.256	332.915	281.919	0.0004	1178.4904
Grand Total	622.4493	595.4293	672.4421	604.0735	0.0021	2494.3963

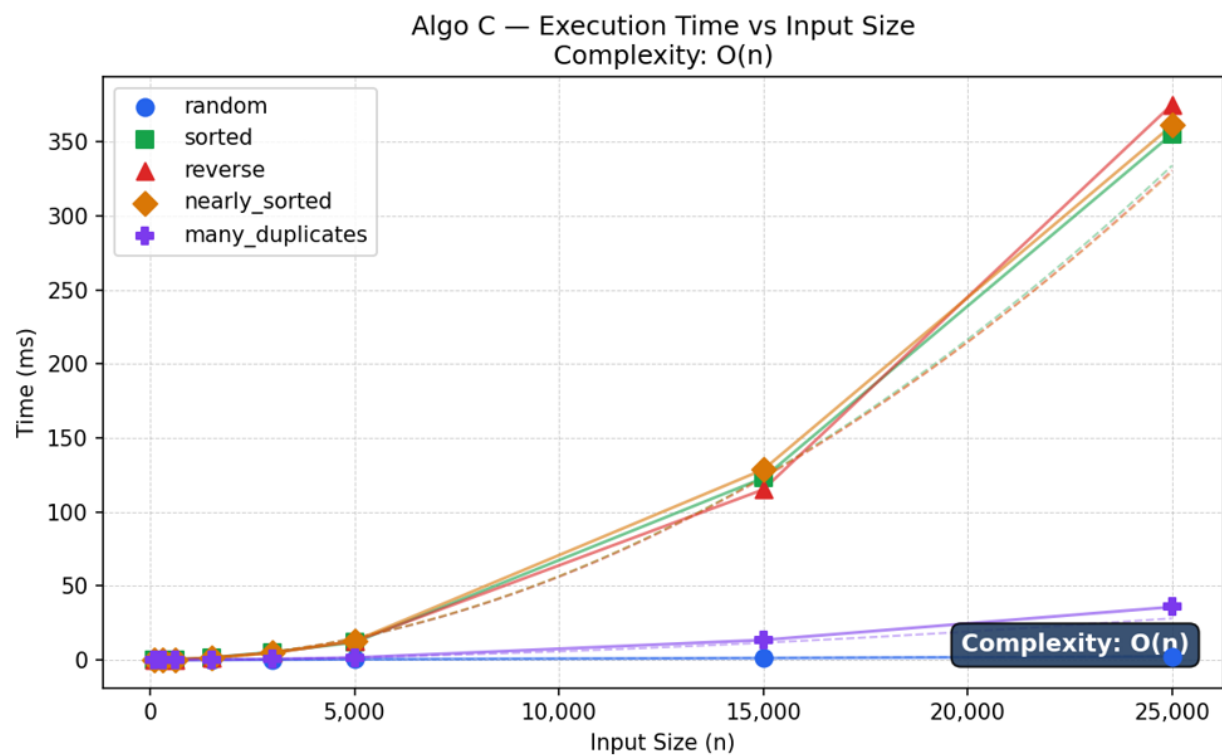
Algo B — Execution Time vs Input Size
Complexity: $O(n!)$



algo C

Sum of time_ms	Column Labels					Grand
Row Labels	many_duplicates	nearly_sorted	random	reverse	sorted	Total
100	0.00278	0.00708	0.00264	0.00694	0.00654	0.02598
300	0.0158	0.07898	0.01242	0.06408	0.06928	0.24056
600	0.03174	0.31164	0.03348	0.27212	0.2594	0.90838
1500	0.16242	1.42806	0.09566	1.60858	1.63292	4.92764
3000	0.62334	4.76788	0.1811	4.89144	5.34902	15.81278

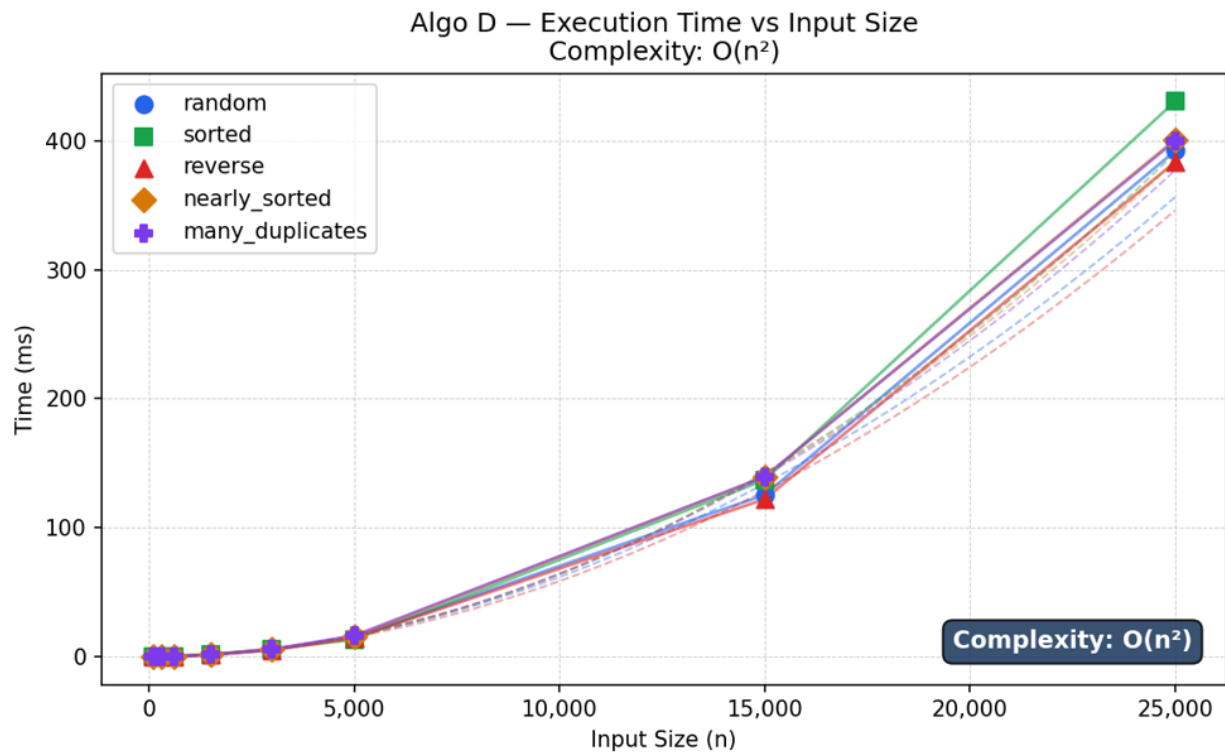
5000	1.5923	12.9012	0.44618	12.6005	11.8563	39.39648
15000	13.4933	128.652	1.28834	115.154	122.948	381.5356
25000	35.8337	361.229	2.21204	375.292	355.426	1129.993
Grand Total	51.75538	509.3758	4.27186	509.8897	497.5475	1572.84



algo	D
------	---

Sum of time_ms	Column Labels					
Row Labels	many_duplicate s	nearly_sort ed	random	reverse	sorted	Grand Total
100	0.00834	0.0072	0.00828	0.00726	0.00668	0.03776
300	0.08766	0.08598	0.08342	0.06866	0.0696	0.39532
600	0.25668	0.24256	0.3067	0.3075	0.25528	1.36872
1500	1.66956	1.74114	1.9553	1.37124	1.70414	8.44138
3000	5.82594	5.73736	5.23086	5.22764	5.942	27.9638

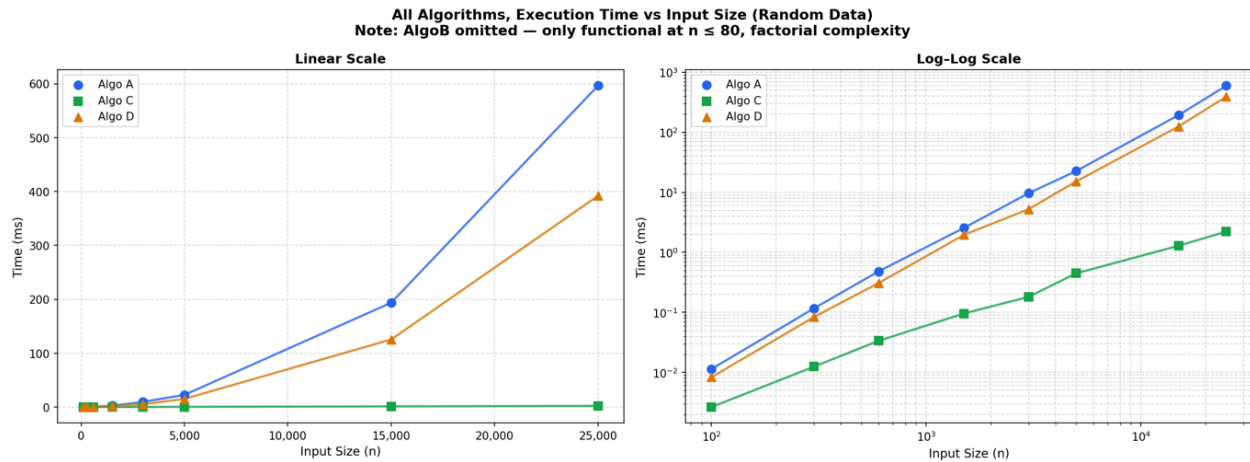
5000	16.3477	15.6772	15.1605	14.5155	13.3535	75.0544
15000	139.523	139.428	125.437	122.095	137	663.483
25000	399.721	400.845	392.054	383.372	430.437	2006.429
Grand Total	563.43988	563.76444	540.2360	526.964	588.768	2783.1733
			6	8	2	8



algo (All)

Sum of time_ms	Column Labels					
Row Labels	many_duplicates	nearly_sorted	random	reverse	sorted	Grand Total
4	0.0009	0.0011	0.0019	0.0011	0.0006	0.0056
15	66.8515	54.2957	70.5121	65.072	0.0003	256.7316
20	88.1509	83.3855	90.1121	83.8024	0.0004	345.4513
50	186.046	175.491	178.901	173.279	0.0004	713.7174
80	281.4	282.256	332.915	281.919	0.0004	1178.4904
100	0.02262	0.02484	0.02228	0.0382	0.02298	0.13092
300	0.23932	0.30546	0.21198	0.25148	0.27124	1.27948
600	0.69446	0.98396	0.82082	1.06024	0.89958	4.45906
1500	4.9555	5.67226	4.60448	5.2898	5.91628	26.43832

3000	14.75096	19.8161	15.0539	19.16794	20.72178	89.51068
5000	41.3485	50.4693	38.22658	49.343	48.1427	227.53008
15000	363.4933	478.241	320.41934	435.711	470.491	2068.35564
25000	1039.1157	1424	990.62704	1343.423	1373.459	6170.62474
Grand Total	2087.06966	2574.94222	2042.42852	2458.35816	1919.92666	11082.72522



5. Analysis

AlgoA $O(n^2)$

From $n=5,000$ to $n=25,000$, random data goes 22.6ms to 596ms, that's $26\times$ slower for a $5\times$ input, which matches $O(n^2)$ almost perfectly. All five patterns give similar times at every n , meaning the algorithm does the same work regardless of input order.

AlgoB $O(n!)$

At $n=4$ it ran in 0.002ms, at $n=15$ it was already at 70ms, a $37,000\times$ slowdown for $3.75\times$ more input. Nothing polynomial does that. Sorted data always finishes near 0ms, so there's clearly an early-exit for already-sorted arrays. Everything else was untestable past $n=20$.

AlgoC $O(n)$ average, $O(n^2)$ worst case

On random data, $n=5,000$ to $n=25,000$ gives 0.446ms to 2.21ms, exactly $5\times$ for $5\times$ input, that's $O(n)$. But on sorted and reverse data the same jump gives $30\times$, squarely $O(n^2)$. The algorithm is extremely fast on random and duplicate data but completely falls apart when the data has order.

AlgoD — $O(n^2)$ Same growth as AlgoA: $n=5,000 \rightarrow n=25,000$ on random gives 15.2ms $\rightarrow$ 392ms, a $26\times$ increase. Consistently about 35% faster than AlgoA across all patterns, but the same quadratic curve.

6. Conclusion

AlgoA and AlgoD are both $O(n^2)$, same growth curve, AlgoD just has a smaller constant. AlgoB is unusable past $n=20$ due to factorial-level growth, with the only exception being sorted data where an early exit saves it. AlgoC is the most nuanced: genuinely $O(n)$ on random and duplicate data, but it degrades to $O(n^2)$ on sorted and reverse inputs, so it's only fast when the data is unordered. The main lesson here is that best-case complexity isn't enough; knowing when an algorithm breaks down matters just as much.